

How probability proofs are completed

Shuo Deng

Kenneth W. Shum*

Revision 28

15 September 2026

Results updated through 14 September 2026

An evaluation of the ProbabilityTheoryFormalization project

Revision 28 · 15 September 2026 · Results updated through 14 September 2026

Abstract

How do AI agents complete probability proofs and make their results usable by subsequent tasks? This report evaluates ProbabilityTheoryFormalization through earlier reviews and code revisions, together with two batches of three configurations on eleven task groups. It examines mathematical outcomes, the observed roles of review and collaboration, and the reliability of completion judgments. [1,4-12,24,41]

The experiment compares the prescribed workflow A, a Sol author B with access to references and assistance, and Astra coordination of Sol authors and checking services C. Under a common mathematical-outcome criterion, A and B complete 10/11 groups and C completes 11/11 in the first batch; all three complete 11/11 in the second. Code inspection and caller checks establish that the three deliveries originally rejected in the second batch complete the required mathematical content. A has the highest median solving time in both batches without a corresponding coverage advantage. Results are reported separately by batch, retaining differences in task instructions and execution arrangements. [13,14,38,41,42]

Mathematical cases explain how these outcomes developed: review prompted a centered-moment bound from raw moments; an assistant's inversion proof entered uniqueness and later applications; and two separate repairs under explicit instructions completed the eventual-integrability scope omitted from old Scheffe deliveries. Historical code traces density derivation from a probability model, assembly of local limits, and caller revision. Historical label growth is examined separately from comparable mathematical progress, while build and semantic checks of saved code supply further direct evidence. [4-11,17-20,25,27,43]

Second-batch process analysis checks 4,054 actual tool calls across all 33 runs, separately records 179 tool-discovery calls, and constructs 66 process records from each run's first retrieval and first diagnostic. It connects material, feedback, code revisions, and final use, showing function-representation and index adaptations, different outcomes of local trials, and breaks in returning checked code to authors. Recurring tool problems also led to startup-information and parameter-checking prototypes, each with functional validation. [45,46]

The findings support three priorities: review the mathematical scope required of public theorems, check that collaborative work reaches its user and enters the proof, and report completion of equivalent implementations alongside actual effort. Execution processes explain how work advances; common task acceptance

*Corresponding author: wkshum@cuhk.edu.cn.

and checks of historical objects provide a traceable basis for checking the correspondence between progress judgments and the mathematical content being evaluated. [21-23,40,42,43]

Reading guide. Section 1 introduces the project and questions, Section 2 explains evaluation, and Section 3 reports outcomes and resources. Section 4 explains the mathematical work required for complete results; Section 5 follows retrieval and feedback into revision and actual use; Section 6 discusses the evaluation evidence and draws conclusions. Detailed cases and historical analyses appear in Appendix R, uniform-start methods and individual records in Appendix S, and evidence locations for the expanded cases in Appendix T.

1 How the project organizes and maintains proofs

ProbabilityTheoryFormalization is the object of this evaluation: a project for turning textbook mathematics into proofs that can be reviewed, used by later tasks and maintained as dependencies change. A successful build, a passing opinion and a completed textbook task are related but different accomplishments. The report asks three questions:

1. **What mathematics has been completed and made usable?** Does the public theorem express the intended claim, discharge its required proof work and connect to the results and callers on which its use depends? 2. **How do review, assistance and control affect the work?** Which observed decisions repair proofs, validate existing work, leave a scope gap or interrupt delivery, and what information was available when those decisions were made? 3. **How trustworthy are the completion and progress judgments?** Which records refer to distinct opinions and comparable mathematical objects, and what do the resulting labels, code checks and resource measurements actually establish?

Historical reviews and code revisions, and the A/B/C task comparison, answer different parts of these questions. Their relationship is set out in Section 2; they are not combined into one completion rate. This section first explains the documented project design. Evidence of its actual mathematical work, operating failures and maintenance comes from the dated studies that follow, rather than from the presence of a component in the design alone. [1,4-12,26,34,35]

1.1 From a probability textbook to a formal proof library

ProbabilityTheoryFormalization, formerly ToyApollo, develops AI-assisted formalization of a probability textbook. Its goal is to turn textbook definitions, statements and arguments into Lean declarations and proofs that express the intended mathematics and can be used by later chapters. Lean checks the formal proofs; Mathlib supplies the general mathematical library available to them. [26,34]

Work begins with the textbook material. The supplied text is organized into numbered subsections or exercise sections. A task plan identifies the statements to formalize and the earlier results they may use, including explicit references in the textbook. The program assembles the source, requirements, permitted dependencies and working draft into a task package for the author. During implementation, the author may search the allowed library and develop additional supporting lemmas. [34,35]

The project separates mathematical decisions from software checks. AI agents choose proof routes, write code and assess mathematical correspondence. Python programs prepare tasks, invoke checks and enforce conditions for accepting code. Lean checks the code, while SQLite records task status and retained build and review evidence. Drafts are kept separately from the **canonical corpus**, the project's official Lean proof library. Project acceptance brings reviewed work into that library. [34]

In this report, an **agent** is an AI execution session in Codex, with a configured model, conversation context and tools. An **author** writes proofs; a **helper** supplies code or mathematical analysis; a **reviewer** checks statements and proofs. These are AI roles, rather than the human authors of this report.

1.2 Writing, reviewing and accepting a proof

The program gives an AI author the textbook source, task requirements and permitted earlier results. The author writes Lean code and checks it with Lean. A separate AI reviewer then assesses whether the formal statement and proof satisfy the source and task requirements. Work needing correction returns for repair or diagnosis. Before reviewed code enters the official proof library, the program checks that the passing review still applies to the code and materials being accepted. This is the documented process; the later cases show where actual execution diverged. [34,35]

Lean and the AI reviewer answer different questions. Lean checks a proof against its formal statement. **Semantic review** asks whether that statement expresses the required mathematics, whether essential proof work has been completed rather than assumed, and whether listed callers can use the result correctly. A passing Lean check alone cannot establish correspondence with the textbook.

Stage	Responsible actor	Result
Prepare the task	Program	Source, task requirements, permitted dependencies and a working draft are assembled.
Write and check the code	AI author, using search and Lean tools	A candidate proof (the current code version), or diagnostics guiding the next edit.
Review the mathematics	Separate AI reviewer, invoked by an agent managing the workflow or by a human operator	An opinion on the statement, proof responsibilities and listed callers.
Repair or clarify the problem	AI author; a separate read-only diagnoser when needed	Revised proof code, or a clarified statement and proof route.
Accept the reviewed code	Program	A valid review is matched to the current code and requirements; code enters the official library and the operation is recorded.

An ordinary proof gap, such as a missing lemma in an accepted approach, returns to the author for repair. A mismatch with the source, an essential conclusion hidden in an assumption, or an unclear proof route can instead require a **read-only diagnosis**: an AI diagnoses the problem before proof editing continues. It does not write the proof or replace the reviewer. The workflow also blocks resubmission of an unchanged candidate after a semantic failure. Section 1.4 explains the risk-triggered **Math Gate**, which checks a proposed statement and proof route before authoring proceeds. [34]

Request preparation is not the same as execution. For example, `review-now` prepares review material and identifies the next action; an agent managing the workflow or a human operator must still invoke the reviewer. Similarly, helper-produced code must be exported from its workspace and returned to the role that can use it. A completed service call therefore need not mean the author received a usable result. The case analysis checks what was actually returned and read. [19,34]

Finally, the program checks that the review concerns the current candidate, auxiliary code, source and requirements. The records call this **review binding**. It prevents an opinion about one version from approving another. If applying the code fails, the prior official file is restored. A failed re-review of an existing official proof normally preserves that code and records a repair need. These checks maintain the relation between a decision and its evidence; they do not make the AI’s mathematical judgment infallible, as the L3 case in Appendix R.1.2 shows. [34,39]

1.3 What it means to complete a proof

A **source obligation** is mathematical work required by the task. A **public premise** is an assumption a caller must supply. Moving a required intermediate conclusion into such a premise leaves that proof to the caller. A **candidate** is a particular code version. An earlier project revision illustrates this distinction.

The historical inversion task `thm_9_5`, which connects interval probabilities to integrals of a characteristic function, shows why a valid conditional proof can still leave work unfinished. Its early candidate already contained substitution, pointwise cases, dominated convergence and rescaling. An assumption called `h_spine` supplied three analytical ingredients: integrability, justification for exchanging integrals and the Dirichlet-integral limit. Successive candidates moved those duties into the proof. [1]

Revision stage	Mathematical work already present or added	Work still supplied by the caller
Initial conditional proof	Main argument conditional on its analytical ingredients	Integrability, integral interchange and the key limit
Next candidate	Product integrability from bounds and finite measure	Interchange and the limit
Following candidate	Interchange for nonnegative truncation parameters	The Dirichlet-integral limit
Completed assembly	Limit from error bounds; previously proved ingredients assembled	<code>h_spine</code> removed

The restriction to nonnegative truncation parameters matters. The final proof sends the parameter to positive infinity, so it does not need the stronger all-parameter auxiliary statement initially planned. Narrowing that internal lemma avoids unnecessary work without weakening the textbook target. Retaining a required ingredient in the public `h_spine` premise, by contrast, leaves real work outside the delivered theorem. Earlier reviews recognized intermediate progress that a sequence of non-pass labels alone would hide. [1]

1.4 Mathematical route checks and task scheduling

A **parent task** owns the original textbook claim. The **Math Gate** checks a natural-language proof route and proposed Lean statement when there is source mismatch, core proof work shifted into public assumptions, or repeated failure without a clear route. Most small tasks do not use it. A `go` permits planned parent-task writing; a `stop` suspends ordinary writing under that plan until the reported issue is addressed. The instruction determines the scope of a pause; the label alone does not establish that all support work is prohibited. [26,34]

For multiple tasks, batch planning reads the recorded state and dependencies to distinguish work ready for authoring, work needing review or route checking, and work blocked by an upstream task. It can organize a worker queue around downstream dependencies and potential work conflicts. The plan identifies the next actions; the agent managing the workflow invokes the corresponding roles, and each task still follows its own acceptance path. An empty ordinary-author queue can therefore mean that a reviewer or diagnoser must act next, rather than that all work is finished or the user must intervene. [34]

1.5 Library interfaces and maintenance after acceptance

The same textbook concept can have different Lean representations in the project and in Mathlib. Importing a theorem expressed with the project’s expectation, for example, does not by itself convert it into a theorem about a Mathlib integral. For important shared concepts, the documented policy is to introduce the textbook

definition and basic properties, prove a **translation theorem** connecting that interface to Mathlib, and then usually use the Mathlib form in later work. This is a policy for recurring mathematical interfaces, not a required cycle for every local lemma. Existing runnable files are kept unless a concrete reuse problem calls for an interface repair. [35]

Dependency selection follows actual use. Mentioning expectation in a source passage supplies mathematical context; it does not automatically require importing the project's expectation definition. An import is justified by the declaration, earlier theorem or translation the proof needs, and the dependency trail records that reason. General library infrastructure is legitimate support. If an auxiliary theorem carries the assigned task's mathematical content, however, review must inspect it as part of that task's proof, not exempt it because it is named a bridge. The variance-expansion comparison in [Appendix R.1.1](#) and the Gamma/Dirichlet history in [Appendix R.3.1](#) examine this distinction in concrete proofs. [35]

Acceptance does not end maintenance. A changed definition or theorem signature can require callers to provide different arguments or assumptions. The task and dependency records identify affected work; builds and review check the revised interfaces and their use. SQLite keeps a rebuildable index of the underlying evidence, separating an earlier passing review, coverage of the current code and support, and a working candidate that still needs attention. A change requires the relevant coverage to be checked again, not merely retention of a pass label. The total-variation repair in [Appendix R.3.2](#) shows this obligation concretely: the source's probability-measure condition had to reach both the definition and its callers. [7,34]

2 Evaluation evidence and methods

Two bodies of evidence address the project questions in [Section 1](#). **Historical reviews and corresponding code revisions** show how proof responsibilities, public statements, shared support and callers changed during the project's development. They also allow the study to reconstruct distinct judgments, qualify comparisons of mathematical targets, examine selection after failure and check saved code beyond its acceptance labels. These analyses have their own questions, methods and findings in [Appendix R.3](#) and [Appendices C, D and L](#). [1,4-11,32]

The eleven-group A/B/C comparison examines the project's prescribed workflow alongside two alternative ways of organizing work on the supplied tasks. Deliveries and resource accounts establish what each executed arrangement produced and expended; execution records explain particular repairs, missed checks and interruptions. [Sections 2.1–2.4](#) specify that comparison, and [Sections 3–5](#) report its findings. The experiment does not replace the historical evidence about maintenance, nor does historical acceptance supply additional experimental successes. [12-14,24,31,38]

Trajectory analysis connects work to outcomes in both bodies. It follows the mathematical duty, the opinion or instruction, the observable response, and the code and delivery subsequently assessed. The available detail differs between historical records and experimental logs; receipt, use or motivation is not supplied where the record cannot establish it. [Section 2.5](#) explains the reconstruction and its coverage. The questions organizing this report are separated from the original experimental motivation below and from the retrospective analyses subsequently performed. [9,10,17-20,24,25,27,29,40]

2.1 Why compare A, B and C?

Completing a probability proof requires writing code and deciding when to check it, how to respond to errors and when to submit. The project prescribes review and repair procedures for these decisions. The experiment examines whether those arrangements help agents deliver complete proofs, comparing them

with two autonomous alternatives: a Sol lead author organizing its own work, and Astra coordinating Sol authors and checking services. All three work on the same task groups. B and C are experimental alternatives, not later production stages of the project. We compare completion, elapsed solving time and recorded model activity. [12,14]

A follows the prescribed project workflow. A Sol author chooses proof routes, searches the library and edits code; other Sol sessions perform checks. The workflow sets review and repair requirements and conditions for accepting code into the maintained library.

B lets the Sol lead author organize the work. It receives general work guidance, can consult project workflow and review documents, and can request other Sol sessions to help write or review code. The lead author decides how to use these resources, whether to request checks and when to submit. The experiment does not require it to complete A’s prescribed sequence.

C lets Astra organize Sol’s work. The Astra coordinator can read tasks and code, give a Sol author specific mathematical advice or editing instructions, and choose its next action from the author’s results and checking feedback. It knows the available services and how to invoke them, but does not receive the full internal checking instructions. Calling checks is its choice; checkers still work under instructions fixed by the backend. [12,18,38]

Configuration	Who organizes the work?	What assistance is available?	How are checks arranged?
A: prescribed workflow	Sol author under project requirements	Project-role instructions and AI checks	Workflow sets review and repair requirements
B: Sol-led organization	Sol lead author	Project references; other Sol authors or reviewers	Lead author chooses calls and follow-up
C: Astra coordination	Astra coordinator; Sol author implements	Tool descriptions; Sol author and fixed checking services; returned feedback	Coordinator chooses calls and follow-up

All these roles are AI agents. Sol and Astra denote the recorded gpt-5.6-sol and gpt-6-astra configurations. Authors and checking roles use Sol; only C’s coordinator uses Astra. All use medium reasoning effort: medium names the reasoning setting, not a model-capability tier. B is an autonomous configuration with document access and assistance; the experiment does not include a separate minimal baseline given only the problem without project documents or other-agent help. C changes the decision-making model, role structure and information access together, so the experiment compares three complete arrangements. [12,18,31]

Within this comparison, trajectory analysis asks when a prescribed or voluntarily requested check identifies the missing mathematical work, when it incorrectly clears a result, how a coordinator uses returned feedback, and why work continues or stops. For the same task, it first checks whether configurations encounter the same difficulty, then traces who identifies it, whether the opinion arrives and how code changes. Appendix R.1–R.2 use these sequences to explain specific strengths and failures of the executed arrangements, rather than treating the completion totals as a measure of review quality.

2.2 What information do agents receive, and how does a task proceed?

Each configuration receives textbook source, task requirements, the permitted mathematical library and designated prior results. Agents must submit theorem statements, proofs, necessary auxiliary code and an explanation of how they meet the task. A **delivery** comprises this code and explanation. [12]

An **outer experiment agent**, also an AI agent in Codex, starts and manages a run. A, B or C organizes solving with the supplied materials and time allowance, then delivers its work. Fresh reviewer sessions evaluate the delivery against common criteria. Internal review, acceptance into the project library and experimental acceptance are recorded separately; Section 2.4 explains scoring.

The outer experiment agent and C’s Astra coordinator have different responsibilities: the former manages experimental runs, while the latter organizes proof work inside C. The outer agent can intervene in any configuration, for example by stopping further activity. Section 2.3 and individual cases describe these actions and their consequences. [31,38,39]

The materials serve different purposes. **Workflow documents** explain how the project organizes writing, review and repair. **Review documents** explain what checks should address, such as extra assumptions or missing parts of the required conclusion. **Tool descriptions** explain which services can be requested and how to invoke them. **Checking feedback** identifies problems in the current code or gives a passing opinion.

A’s authors and checkers receive their respective project-role instructions. B’s initial prompt gives general advice to plan, search, check and seek help as needed; workflow and review references are available for active consultation. Availability does not establish that a particular run reads them. C’s coordinator can read public tasks, code and feedback, but does not receive the full internal general-review and mathematical-check instructions. Feedback may nevertheless reveal some criteria. C therefore chooses how to organize work; the coordinator does not freely set the task requirements or the backend’s checking instructions. [38,39]

B’s lead author can operate a terminal, edit code and compose requests to other Sol sessions. C separates coordination from execution: Astra reads code, assigns authors and requests checks through tools; the Sol author operates a terminal and edits code. The coordinator can give specific proof advice, but cannot rewrite a checker’s internal instructions through the fixed checking tools. Both configurations’ internal reviews are part of solving; common experimental evaluation follows submission. [38,39]

The experiment selects eleven groups covering different mathematical work and organizational questions. Some groups require several connected results. L, M and H retain the planned workload groupings; the table specifies the actual tasks, and subsequent cases use these identifiers. [12,31]

Group	Mathematical task
L1	Construct innovations and prove their partial sums form a martingale
L2	Textbook 11.4 variance additivity and 11.5 weak law
L3	Scheffé’s lemma, density convergence and associated distributional conclusions
L4	Total-variation formulas and distribution calculations
L5	Construct a maximal coupling
L6	One- and two-sensor linear minimum mean-square error
L7	Coordinatewise vector convergence and continuous mapping
M1	Stopped processes and stopped expectations
M2	Strong law under fourth-moment conditions
M3	Skorokhod representation on a common probability space
H1	Characteristic-function inversion, uniqueness and two applications

2.3 Which runs enter the comparison, and what changed during execution?

Early B and C trials exposed two execution problems: some promised documents or tools were not actually supplied, and some checking opinions or helper code were generated but did not reach the agent deciding what to do next. The execution programs were subsequently repaired. Because these problems affect usable information, the main comparison and early trials are reported separately. [31,38]

One **solving run** spans initial dispatch, author revisions, assistance, checks and continuation under the same clock. Development records contain 41 such runs. The main panel retains one delivery for each configuration and task group, giving 33 entries. A contributes nine earlier runs and two later runs for H1 and M3. B and C each contribute eleven runs after repairs to their execution programs. Early B and C trials support analysis of failures and feedback use. Actual calls still require inspection: repairing a program does not establish that every later call succeeds. [12,13,31]

A run can leave multiple deliveries. In recorded order, aggregation preferentially retains a complete delivery with a common evaluation; if none exists, it retains the first evaluated delivery. This includes A-H1's completion after continuation under its original clock. The rule is established by the retrospective aggregation code; no record fixing it before execution was recovered. Appendix N.1 retains the exact selection steps and run mapping. [31]

Textbook source and task sheets match across configurations in all eleven groups selected for common evaluation. Lists of permitted prior results also match except for an additional, unimplemented continuous-mapping target listed for A-L7. Materials, tools and interventions during solving have the following specific differences. [31]

Execution difference	What happened?	How should outcomes be interpreted?
B's documents and tools	Some promised capabilities were absent in early trials; later provision was verified	Main-panel B uses the repaired arrangement; early trials are separate
B/C result delivery	Later programs repair some failures to return opinions or helper code	Analysis follows what the decision maker actually receives
Additional task guidance	B's final reference package removes a mandatory named L4 interface and M3/H1 module-name hints; A-L4 received a related conditional instruction	Report the concrete guidance; its effects have not been quantified
A-L7 external stop	Further editing, help and new review were forbidden with about 61 minutes left	Original delivery reflects an early stop; resumed work is separate

Extending C to different tasks also changed which tasks could launch and their time-allowance settings. The selected C runs share core backend and role-prompt contents. Appendix O.1 explains the restrictions, organizational hints and actual exposure, as well as the scope of this implementation commonality. The main text retains conditions that affect interpretation, and subsequent cases examine their role through code and feedback. [38]

On 13 September, A-L7 resumed its original author on a copy of the sealed working state, with explicit permission to deliver a reasonable repaired statement. Its delivery enters the completion results in Section 3.1, and its additional time enters the cumulative measure in Section 3.2. The original-round records and metrics remain alongside them; the historical inventory of 41 runs is unchanged. Appendix R.2.6 explains the continuation conditions and proof changes. [39]

The second batch completes one run per configuration on the same eleven groups in a registered order, for 33 main runs. Source mathematics, supplied libraries, model names, reasoning settings and allowances are unchanged. Five task groups receive added acceptance explanations; parts of A's re-review and revision

instructions, some reference material, and client and tool routing also change. The first batch had already been reassessed under corrected criteria: rescoring itself is not a change to solving conditions, whereas advance instructions change the solver input. Results are therefore reported by batch. [Appendix Q.1](#) gives the differences. [\[41\]](#)

2.4 Completion evidence and post-submission common evaluation

Calling an experimental task complete requires a judgment about the delivered mathematical scope, not just a successful command or the author’s decision to submit. Lean checks the statement actually written; a semantic reviewer assesses its relation to the task; project application validates the review’s correspondence and eligibility before accepting code. For the task comparison, separate post-submission reviewers make the reported completion judgment under the common procedure below. The following evidence types therefore answer different questions:

Evidence	Question it answers
Build	Does the written code check in the recorded environment?
Semantic review	Does the reviewer judge the statement and proof to meet the source obligations?
Binding and production acceptance	Has the project applied an eligible review to the required code and current review basis?
Post-submission common evaluation	Does the experimental delivery meet the criteria adopted for this experiment?

Section 3 uses a common mathematical-outcome criterion across both batches: the delivery must cover the full target, allowing equivalent routes and composable interfaces. Independent caller checks verify redundant parameters directly entailed by the original assumptions and permitted library. Compliance with a prescribed route is recorded separately. Original common reviews supply existing evidence; subsequent reconciliation corrects inconsistent judgments while retaining those reviews. The original review procedure follows below; [Appendix Q.2](#) gives the reconciliation rules and three changes across 66 items. Historical records retain their dated meanings, and experimental completion remains distinct from production acceptance. [\[13,26,42\]](#)

Common evaluation is the experiment’s assessment after submission, separate from help and internal review during solving and from production acceptance. The original endpoint procedure requested two fresh, read-only semantic opinions on the source, task, candidate, permitted upstream material and recorded technical checks. Configuration metadata was withheld from the evaluator-facing request. Matching pass or fail votes completed the decision; two inconclusive votes left it unresolved. Different votes triggered at most one fresh adjudicator, who read the same materials and the two opinions and resolved their substantive disagreement rather than choosing by confidence. All original, consistency and targeted endpoint calls use the recorded Sol medium configuration. [\[30\]](#); [Appendix N.6](#)

A group decision concerns the required complete delivery. Passing the coordinate-convergence theorem alone does not complete the two-target vector-convergence group, and a verdict on explicitly partial material describes that material rather than the missing task. Invalid result formatting is also separated from mathematics. In the representation-theorem case, a reviewer wrote a pass but used an object where confidence had to be a number; the evaluation software recorded an inconclusive execution result and the procedure proceeded to adjudication. [\[22,28,30\]](#)

Reassessment then examined inconsistent interpretations in variance expansion, Scheffé’s lemma and Cauchy scope. Each candidate received separate initial opinions under the stated readings. A necessary ad-

judication could be triggered by disagreement on an obligation, an invalid result, or inconsistent treatment of corresponding behavior across candidates. These adjudicators could see the new initial opinions and a cross-candidate consistency summary; they were not blind to that later evidence. For the inversion group, the original submission explanations were supplied symmetrically after the six initial reviews, when the anonymous code-only view proved insufficient for a documentation judgment. [23,30]

The corrected acceptance for the 12 September comparison adopted four explicit readings: permitted general expansion for variance additivity, the task’s eventual-tail formulation for Scheffé, the textbook’s standard-Cauchy core, and reasonable disclosed repairs of the deficient continuous-mapping assumptions. At 09:22 on 12 September, existing opinions were reused, determinable format mismatches were repaired and outcomes were recomposed under those criteria. The two additional continuous-mapping opinions had already been obtained in the 05:17 supplement. This was a retrospective criterion correction, not new solver work. Appendix R.1 gives its mathematical basis and retains the original outcomes. [13,15,22,30]

Appendix N.6 states the stage-specific rules, evaluation inputs and masking limits. Here **independent** means the documented separation of reviewer sessions and roles; adjudicators deliberately receive earlier evidence, and using separate sessions of the same model does not establish independent model errors.

The 13 September A-L7 continuation is a separate delivery and assessment. Its two fresh evaluations distinguish correctness of the disclosed repaired statement from literal compliance with the original statement. Appendix R.2.6 and Appendix O.2 report that outcome and its additional resources; the completed continuation is included in Section 3.1. [39]

2.5 Following information, decisions and proof changes

Trajectory analysis asks how a task reaches its result. For each focal case, we connect the mathematical requirement, information actually received by the author or coordinator, the next action, changes to the declaration and proof, and the version eventually assessed. Historical cases use the same questions across reviews and code revisions, with additional checks that the earlier and later records concern comparable mathematical targets. [8-11,17-20,25,27,29]

The central comparison is between the work before and after a consequential event. A review may lead to a new derivation, support submission of unchanged code, or fail to reach the author. A helper may produce a valid proof that never enters the delivered result. We inspect tool returns and reading records, compare code, and follow actual calls to reused results to distinguish these processes. When a link cannot be recovered, its contribution remains unconfirmed; the observed sequence can still explain the work around it. Appendix R.2 presents those findings. [17-20,24,25,27,40]

The cases were selected from investigated repairs, assistance and execution problems. They support explanations of particular processes. Broad log extraction covers all recorded solving runs, but assessing the correctness and use of feedback requires further reading. A separate assessment applies the same detailed criteria to six of 127 indexed internal services; the other case studies have their own documented scope. These sources do not establish an overall rate of correct-feedback adoption or mathematical repair. Appendix N.4 gives the units and coverage, G.18 gives the six assessments, and M.2 gives log-processing rules. [16,24]

A separate study reviews each of two fixed old L3 candidates under the original instruction and an explicit tail requirement, with three fresh reviews per condition: 12 in total. Each old candidate also receives one explicitly directed repair. Diagnostic code and repair instructions are separately frozen; these runs are separate from the 33 main runs. Appendix R.1.6 reports the results and Appendix Q.4 provides inputs and decisions. [43]

A further analysis covers all 33 second-batch runs using the same starting-point rules: the earliest mathematical retrieval and the earliest proof or interface diagnostic in each run, followed through later actions, local results and final code. This yields 66 records. The starts concern the same need in 25 runs and are summarized separately. Chapter 5 reports this analysis; selection rules, per-run records and reviews appear in [Appendix S. \[45\]](#)

3 Experimental outcomes and effort

This chapter reports acceptance outcomes, solving time and tool activity for eleven task groups under three configurations.

3.1 Completion in the two batches

Evaluation is complete for all 66 main deliveries. The common mathematical-outcome assessment records 31 completions in the first batch and 33 in the second. The table gives each result.

Group	Batch 1 A	Batch 1 B	Batch 1 C	Batch 2 A	Batch 2 B	Batch 2 C
L1	Complete	Complete	Complete	Complete	Complete	Complete
L2	Complete	Complete	Complete	Complete	Complete	Complete
L3	Not complete	Not complete	Complete	Complete	Complete	Complete
L4	Complete	Complete	Complete	Complete	Complete	Complete
L5	Complete	Complete	Complete	Complete	Complete	Complete
L6	Complete	Complete	Complete	Complete	Complete	Complete
L7	Complete	Complete	Complete	Complete	Complete	Complete
M1	Complete	Complete	Complete	Complete	Complete	Complete
M2	Complete	Complete	Complete	Complete	Complete	Complete
M3	Complete	Complete	Complete	Complete	Complete	Complete
H1	Complete	Complete	Complete	Complete	Complete	Complete
Total	10/11	10/11	11/11	11/11	11/11	11/11

The first-batch A and B L3 deliveries require integrability at every index and lack the tail argument for eventually integrable sequences.

Original second-batch acceptance recorded 30 passes and rejected A-M1, B-M2, and C-L3. The common reassessment examined the existing deliveries individually: A-M1's existence theorem and stopped-expectation conclusions compose on the original input to cover all three required cases; B-M2 already proves the full-sequence conclusion directly, although it does not follow the specified subsequence route; and C-L3's additional premises follow from general library facts on the same input, with a successful independent caller check. All three therefore count as complete under the common mathematical-outcome criterion, with route compliance recorded separately. [Appendix R.1.5](#) explains each correction; [Appendix Q](#) retains the original decisions and reconciliation procedure. [\[41,42\]](#)

A's first-batch L7 includes its authorized continuation, and its second-batch L3 includes engineering recovery. Both additional solving windows enter the costs below. Two separately directed repairs of old L3 candidates also passed; [Appendix R.1.6](#) reports them as separate evidence about repair.

3.2 Solving time on matched tasks

Solving time runs from the first role dispatch to the last role return, including waiting within a run. Overlapping assistance is counted once. Separately resumed solving windows are added; time in storage between windows and independent post-submission evaluation are reported separately. Medians cover all eleven groups in each batch. [14,41,42]

Batch / median cumulative minutes	A	B	C
1	69.78	30.09	26.79
2	41.03	20.21	22.01

The per-task times below are in minutes. Each row refers to the same task group and can be read alongside its completion result in the preceding section.

Group	Batch 1 A	Batch 1 B	Batch 1 C	Batch 2 A	Batch 2 B	Batch 2 C
L1	85.82*	11.90	12.16	15.21	14.84	15.27
L2	14.25	30.09	18.00	16.75	16.30	18.98
L3	69.78	33.15	39.90	44.35	13.65	23.34
L4	46.11	15.89	16.54	41.03	18.79	18.04
L5	76.96	67.61	28.70	32.72	19.70	30.09
L6	34.34	21.98	23.61	24.51	34.13	21.49
L7	73.37	28.99	32.27	21.61	23.99	22.01
M1	136.89*	17.64	26.79	51.91	23.70	24.19
M2	59.54	36.38	36.53	51.17	40.56	22.91
M3	21.13	41.54	15.23	50.24	20.21	18.40
H1	182.16	126.17	103.36	135.59	123.89	86.42

* marks estimates reconstructed from original records. First-batch A-L7 combines 58.86 minutes with a 14.50-minute continuation; second-batch A-L3 adds an 18.71-minute engineering-recovery window. Totals use unrounded seconds. Appendices M, P and Q.3 provide endpoints and sources.

Matched tasks explain differences that a single median conceals. In the second batch, B has a lower median than C, but B is faster on 5 groups and C on 6; their H1 times are 123.89 and 86.42 minutes. A has the highest median in both batches but is fastest on first-batch L2. First-batch L3 times must also be read alongside noncompletion. Instructions, parts of the workflow and software changed between batches, so lower second-batch times are described without assigning them to a single mechanism.

3.3 Tool problems across the runs

Unavailable commands occur in all 33 selected first-batch runs, and file-access problems in 27. A, B and C respectively record 180, 291 and 322 tool returns containing Lean compilation or interface errors. These counts include failed intermediate builds, interface probes and repeated attempts. [Appendix M.2](#) gives the detection rules, and [M.3](#) identifies the affected runs. [24]

A task can pass after several errors and revisions. A-M2, for example, underwent multiple failed builds before the proof was completed and accepted. Error-bearing tool returns describe feedback during the process; completion is judged from the final delivery. [Appendix R.2](#) connects received feedback, code changes and submission records to examine which problems were resolved and which obstructed delivery.

3.4 Model usage and solving time

Time and tokens measure different forms of effort. On first-batch M2, B took 36.38 minutes and recorded 15.237 million input tokens; A took 59.54 minutes and recorded 13.880 million. Both passed, and the faster run used more input tokens. [14]

First-batch token comparisons use the eight groups with complete records in all three configurations; A-L1, A-M1 and A-L5 still have usage gaps. Input already includes cached input, and output already includes reasoning output; these subsets are not added again. Author and internal assistance, post-submission common evaluation, later reassessment and engineering probes are accounted for separately in [Appendix N.5](#). [16,31]

Second-batch recorded input totals are 143.819 million tokens for A, 97.605 million for B and 87.452 million for C; A-L3 recovery adds 6.071 million. A capacity-error call in B-H1 lacks complete usage, so B's total is a known lower bound. Resource counts complement elapsed time; [Appendix Q.3](#) gives per-run usage and known token subtotals for independent evaluation. [41]

4 How mathematical work becomes a complete, usable result

This section asks what mathematics agents completed behind the outcome labels. It examines the input scope of theorems, derivations that belong inside proofs, subsequent calls to collaborative results, and mathematical development across historical revisions. First-batch runs, separate repairs of old candidates, and historical code revisions are identified by source. Section 5 then starts from uniform second-batch observations to follow retrieval and feedback into concrete revisions.

4.1 How was a scope omission accepted and subsequently repaired?

The first-batch Scheffé lemma task (L3) supports subsequent density problems. For a nonnegative function sequence with the given almost-everywhere convergence, the limit f is integrable and the function integrals converge to its finite integral; the target is $\int |f_n - f| \rightarrow 0$. The disputed scope concerns the sequences accepted by the lemma: the original A and B deliveries require integrability at every index, whereas the common mathematical-outcome criterion requires coverage of eventually integrable sequences. The original instructions permitted a tail formulation; the later common evaluation explicitly adopted that scope. This distinction matters when interpreting the decisions made by authors and reviewers at the time. [23,39]

The two input classes differ. On $(0, 1]$, take the first function to be $1/x$ and all subsequent functions and the limit to be zero. Every tail term is integrable, but the first term is not. The old theorem for termwise-integrable sequences cannot be applied directly to this sequence. Completing the task requires selecting an integrable tail, carrying out the integral argument there, and recovering the conclusion for the original sequence because a finite initial segment does not change its limit. This adds a proof for the permitted inputs. [Appendix R.1.2](#) preserves the precise scope and code evidence.

Other problems had already been repaired during review. A initially imposed nonnegativity and measurability of the limit function as additional assumptions. After a general review identified this gap, the author derived the required properties inside the proof. However, the mathematical-route check interpreted termwise integrability as a legitimate encoding of finite real-valued integrals; the general re-review accepted the same interpretation. Its downstream density example was already termwise integrable and could call the old lemma, so it did not test the broader tail inputs. B's self-requested mathematical review also noticed the tail version but treated it as an optional extension, and the author submitted the original implementation. [29,39,40]

This sequence explains how the omission remained: local assumptions had been repaired and the downstream caller worked, so review accepted a lemma with narrower scope. The acceptance program checked that the opinion matched the correct code and materials; it did not make another mathematical scope judgment. Its later checks therefore left the decision unchanged.

To examine that judgment further, the second stage held the two old code versions fixed and reviewed each with the original instructions and with an added tail-scope check, three times per condition. The table distinguishes discussing the omission, requiring repair, and passing the candidate. [43]

Fixed source and instructions	Discussed tail omission	Required repair	Passed
A, original instructions	0/3	0/3	3/3
A, added scope check	3/3	3/3	0/3
B, original instructions	3/3	1/3	2/3
B, added scope check	3/3	3/3	0/3

For A, the first difference is whether review notices the narrower input scope. All original-instruction reviews of B discussed the tail formulation, but two still judged it unnecessary. With the explicit requirement added, every review of both candidates required repair. Recognizing a possible extension and deciding that it belongs to the current delivery are different decisions here. These observations concern fixed cases with three reviews per condition.

Two targeted repairs with separately frozen instructions subsequently filled the gap. A added an interface with an existential cutoff; B used eventual integrability and retained a compatible termwise-integrable theorem. Both used the integral argument only on an integrable tail and then recovered the original-sequence conclusion. They took 7.28 and 6.98 minutes respectively and passed both build and common acceptance. Their instructions were fixed before the diagnostic results returned, so they demonstrate new implementations under explicit requirements. The repairs are reported separately; the original first-batch deliveries remain scored on their original code. [Appendix R.1.6](#) and [Q.4](#)

4.2 How did feedback bring the public theorem back to the original assumptions?

The first-batch fourth-moment strong-law task A-M2 provides another example of changing assumptions. The textbook assumes independent variables with common mean m and a uniform raw fourth-moment bound $\mathbb{E}X_i^4 \leq c$. The preparation summary instead mentioned central moments, and the draft required callers to supply a central fourth-moment bound for $X_i - m$. Delivering the textbook result required deriving the central bound internally from the raw one. [25]

The author first searched for strong-law, independence, and integral-product interfaces, read implementations, and checked declaration types in small files. It then expanded the fourth moment of a finite sum, used independence to eliminate the relevant terms, obtained a tail-probability bound, and applied Borel–Cantelli. Of the first eight builds, seven failed and the eighth passed. The central-moment route was implemented at that point, but the public declaration still left its required central bound to the caller.

General review identified the difference. The author subsequently proved

$$(x - m)^4 \leq 8(x^4 + m^4), \quad \mathbb{E}(X_i - m)^4 \leq 8(c + m^4) \quad \text{when} \quad \mathbb{E}X_i^4 \leq c.$$

The pointwise inequality lets the raw moment control the central moment; the expectation bound then supplies the existing proof. In version eleven, the public theorem accepts the textbook’s raw-moment assumptions and calls the completed central-moment auxiliary result. The corresponding re-review passed,

the final source retains this structure, and the post-submission common evaluation also passed. [Appendix R.2.2](#)

Stage	What the caller must supply	What the author has completed
Version eight builds	A central fourth-moment bound	A strong-law proof based on central-moment expansion
Review identifies the difference	The textbook supplies only a raw fourth-moment bound	The remaining conversion obligation is identified
Version eleven is delivered	The textbook’s raw-moment assumptions	The central bound is derived internally and the existing auxiliary proof is applied

This comparison explains the concrete change associated with review. When the build passed, the code proved its own stated proposition; after the conversion was added, the public proposition used the task’s original assumptions. A new derivation, declaration change, and recheck can all be located after the feedback. The textbook, author implementation, tool returns, and review all participate in this process; a count of reviews alone would not express the change.

The same issue can arise at a time-index boundary. The first-batch B-L1 draft required measurability and integrability at every natural-number time, while the task supplied positive-time conditions and $X_0 = 0$. After receiving advice, the author derived the time-zero properties internally, changed the public interface, and retained the original auxiliary lemma. A-M2 supplied a bound and B-L1 supplied boundary-time properties; both moved required proof work out of the caller’s assumptions. [\[20,27\]](#)

4.3 How did a collaborator’s proof enter subsequent tasks?

The first-batch inversion and uniqueness group H1 contains five connected targets: a bound for an exponential function, characteristic-function inversion, uniqueness through inversion, a Cauchy sample-mean application, and an integer-valued criterion. Here assistance can be examined through the mathematical conclusions actually used. [Appendix R.2.3](#) records A’s same-root completion process and subsequent code calls. [\[17\]](#)

The author initially asked a helper to handle the entire inversion proof while restricting it to read-only work. The return identified the Dirichlet-integral gap but supplied no usable patch. The author then requested the integral limit and a uniform bound, obtained built auxiliary code, and integrated it into the draft. Another helper completed the remaining inversion proof but encountered an export failure. The outer execution layer recovered the artifact, after which the author could read, apply, build, and submit it for internal review.

Producing the mathematical result and making it available to the author therefore occurred at different stages. The inversion argument handles exchange of finite integrals, the pointwise limit of an oscillatory kernel, dominated convergence, and half the mass at each endpoint. Let μ be a probability distribution and ϕ_μ its characteristic function. For $a < b$, it obtains

$$\lim_{T \rightarrow \infty} \frac{1}{2\pi} \int_{-T}^T \frac{e^{-iat} - e^{-ibt}}{it} \phi_\mu(t) dt = \mu((a, b)) + \frac{1}{2}\mu(\{a\}) + \frac{1}{2}\mu(\{b\}).$$

This result enters the uniqueness proof: for two distributions with the same characteristic function, suitable endpoints give equal interval probabilities, and a measure-determination argument yields equality in distribution. The Cauchy-mean application computes the characteristic function of an independent sum

and uses the newly completed uniqueness theorem to identify its distribution. The contribution of the intermediate result can be located through these actual calls.

Completed result	Use in a subsequent proof
Inversion theorem	Uniqueness compares interval probabilities and identifies the distribution
Uniqueness theorem	The Cauchy-mean application identifies a distribution from its characteristic function
Uniqueness file imported by the integer-valued criterion	This branch uses a separate argument through the real part of an exponential, without calling uniqueness

The last row is informative too: a file dependency makes an environment available, whereas a theorem call shows that a mathematical conclusion was used. The integer-valued criterion follows its own proof route, so its import cannot be counted as another adoption of the assisted theorem.

Broad assistance also has examples of completed and adopted work. First-batch B-M1 delegated the stopped-process and stopped-expectation group to a support author; the main author obtained the patch, integrated and built it, and requested mathematical review. Beyond finite stopping, the support implementation handled the limit passage. For a martingale X and an integrable stopping time T , with increments bounded by a constant c ,

$$|X_{T \wedge n}| \leq |X_0| + cT$$

provides the required integrable control. The final target file matches the integrated code. First-batch A-M3 also adopted a helper’s whole-task implementation. In an earlier A-L5 episode, by contrast, a helper produced built code but returned a path outside the author’s container. The author could not obtain the patch and continued a local implementation; the available evidence does not confirm that the helper’s code entered the final proof. [18,19,28]

These cases show read-only advice, auxiliary lemmas, and whole-group implementations. Scope, permissions, and delivery method jointly shape what can happen next. Section 5.4 examines specific code-transfer breaks in the second batch.

Execution recovery can also deliver a proof that already exists. The final theorem signature and proof body for first-batch A-L7 were already present in the sealed working candidate. Once continuation was authorized, the author explicitly disclosed the proposition correction and completed review and delivery. The added window therefore completed disclosure and acceptance of an existing proof. [Appendix R.2.6](#) preserves the states before and after continuation. [39]

4.4 How did historical revisions add usable mathematical content?

Earlier code revisions supplement the observations from individual runs. They span versions of the proof library and its callers. The following accounts identify the mathematical work actually added; these historical records do not enlarge the experimental denominator in Section 3.

Deriving a density from a probability model. An early implementation from normalized Gamma variables to a Dirichlet distribution used an axiomatic bridge. A later revision removed the axiom but directly defined the normalized-variable density as the target formula, leaving the model-to-formula derivation unproved. Subsequent revisions established the product distribution, normalization map, simplex support, projected density, and change of variables, connecting the model to the density formula. This explains why checking axiom names alone does not measure all mathematical progress. [4]

Two downstream tasks used the result while retaining their own coordinate-chart or Beta-branch obligations. Later, the proved density conclusion was added to the public parent theorem, changing where the commitment was exposed. The derivation, extraction of a shared result, and completion of the public interface occurred at different historical points. [Appendix R.3.1](#)

Assembling local results into one global object. The moment-generating-function route requires analytic control on a complex strip, extraction of limits, and identification by characteristic functions. Historical code contained substantive local results, but limits on different compact pieces had to arise from one global function and agree on overlaps. Later construction and compatibility proofs completed this assembly; listing existing local lemmas did not do so. The complete proof preceded the split into support modules. Although a downstream caller later changed only one call, its upstream basis had changed from a private axiom to an internal proof. [6]; [Appendix R.3.2](#)

Making a conditional theorem apply to a concrete model. The saved coupon example studies collecting $m_N = \lfloor (N + 1)/2 \rfloor$ types from N equally likely types, representing the required time by a sum T_N of independent geometric stage waiting times. The June conditional proof permits $N = 2$, when only one type is collected: the sole waiting stage has success probability 1, so the entire row has zero total variance. The configuration also equates the squared normalization scale to that variance while requiring a strictly positive scale. Formal checking derived a contradiction, showing that this configuration had no instance. A separate rejected upstream central-limit candidate from August nevertheless contained substantive Lindeberg and Lyapunov branches; the remaining work connected its concrete independent rows to the probability model used. After that connection, the coupon application still had to replace its actual mean and variance by the textbook’s asymptotic normalization. [8]

For sizes with positive variance, let Z_N use the actual mean and variance. Its relation to the textbook expression is

$$Z_N = \frac{T_N - \mathbb{E}T_N}{\sqrt{\text{Var}(T_N)}}, \quad \frac{T_N - N \log 2}{\sqrt{N(1 - \log 2)}} = a_N Z_N + b_N.$$

Later work established a bounded mean error and the required variance limit, yielding $a_N \rightarrow 1$ and $b_N \rightarrow 0$, and transferred convergence in distribution under the changing affine maps. These are distinct contributions alongside the upstream limit theorem and the concrete distribution connection. The analyzed artifact is the saved independent-stage waiting model; the materials do not separately deliver equivalence to the stopping time of the original sequential coupon draws. [Appendix R.3.3](#)

Changed assumptions also propagate to definitions and callers. After a total-variation revision restored the source’s probability-measure requirement, uses of the expression had to provide the corresponding evidence. A downstream convergence predicate needs both the probability-measure condition and distance convergence. Replacing their conjunction with an implication would make it automatically true when the condition fails. The actual revision passed the needed evidence so that callers continued to express the intended object. [7]

These histories explain what a usable result entails: the proof comes from the stated model, local results assemble into the target, and callers can connect to it. Section 6.3 examines how to keep the objects comparable when counting progress across historical records.

5 From retrieval and revision to actual use

All 33 second-batch runs passed common mathematical acceptance. This section asks: **How do retrieval and feedback enter proof revisions and produce results that are actually used?** Beneath the same final completion outcome, agents followed different searches, interface adaptations, local trials, and collaborative deliveries. The analysis first defines its common observation scope, then connects material, feedback, adjustments, and outcomes for the same proof need.

5.1 Following subsequent work on the same proof need

The analysis starts from complete tool records for three configurations on 11 task groups. Deduplication and pairing yield 4,054 actual tool calls, plus 179 tool-discovery calls. Records retain inputs and returns. A single call can search, modify, and check, so its content must also be read to identify the proof need being addressed. [45]

Two starts are selected per run: the first active lookup of mathematical names, types, or source, and the first proof or interface diagnostic after reading a source file. From each start, the analysis follows queries, edits, checks, and final source concerning the same need. When a helper participates, it also checks what the author received and where it was adopted.

Material examined	Coverage	What this section asks of it
Actual tool calls	4,054	Which actions and returns occurred, and where tool problems appeared
First mathematical retrieval and subsequent work	33	How retrieved material advanced the proof and connected to final source
First proof or interface diagnostic and subsequent work	33	What checking exposed and how the same need was handled afterward

The two starts produce 66 records; in 25 runs they concern the same need, so the two groups are summarized separately. A first retrieval can be routine preparation, and a first diagnostic can come from a temporary query. From these starts, the analysis follows how material becomes usable and how specific problems are handled. The rule covers early work in every run; complete processes for all later needs have not been annotated. Finding a relevant declaration, passing a local check, and final adoption are also recorded separately: the first identifies candidate material whose applicability requires checking, while the latter two require code and checking evidence. Selection rules and review assignments appear in [Appendix S.1–S.3](#). [Appendix T](#) indexes the call order and source locations for the expanded cases.

5.2 What work remains after finding a relevant theorem?

The 33 retrieval records connect to final source as follows. The count follows the need addressed by the first retrieval, including subsequent searches and changes; it describes where that work led.

Connection between the followed process and final source	Records
At least one relevant declaration explicitly used	29
Related implementation or partial result retained	2
First collaborator’s result replaced by a later delivery	1
Connection not yet confirmed	1
Total	33

Connecting distribution-function material to a quantile proof. The second-batch Skorokhod representation task B-M3 needs random variables with specified distributions and the required convergence properties. The support implementation starts from a lower quantile of the distribution function, using monotonicity, endpoint limits, and right continuity to establish quantile properties for the later construction.

The main author first searched for distribution-function and quantile names, broadened the query, and delegated a support implementation. The helper read distribution-function and related representation source and found the needed properties. The obstacle involved two function representations: the project used its own distribution-function expression, while library theorems concerned the library’s function. In the first failing implementation, the helper had already written a lemma showing equality at every real argument. It then tried to replace the target expression directly with `change`; checking still reported that the expressions were not definitionally equal.

Two specific operations differ here. Pointwise equality establishes equal values, whereas that direct target replacement requires the system to recognize the same unfolded expression. The helper later applied function extensionality to the existing pointwise lemma, obtained a function equality, and used it to rewrite the limit, monotonicity, and right-continuity targets. The library theorems could then apply to the original project function.

Order	Recorded action or result
Search and initial implementation	Main author searches and delegates; helper reads source and writes quantile foundations with a pointwise-equality lemma
First check and direct conversion	Function representations mismatch; direct replacement is still rejected
Search and revision	Helper reads supremum interfaces, probes candidate names, and revises bounds; function-representation errors remain
Explicit rewriting and recheck	Pointwise equality yields a function equality for rewriting; other bound revisions are also completed, and the support file passes
Final-source check	Final support source retains the conversion and quantile properties for the subsequent construction

This local process produced checked quantile foundations. The full Skorokhod construction still remained when the helper returned; the run’s ultimate pass belongs to the later delivery. Returns preserve the initial pointwise lemma, failed direct conversion, and successful function rewrite, locating the revision precisely. Supremum proofs also changed during the process, so the whole file’s build success cannot be attributed solely to a function equality.

Turning a reviewer’s route into the author’s proof. Second-batch B-M1 needs to prove that the stopped process remains a martingale. After searching, the reviewer returned a stopped-submartingale interface and an interface combining submartingale and supermartingale properties into a martingale. The return described a concrete route: view the original martingale as a submartingale and stop it, do the same with the negative process, negate to obtain the required supermartingale property, and combine the two. The author actually received this advice, reread the source, and wrote the corresponding implementation.

The first check still failed. The library interface permits stopping times with an infinite value, while the task uses finite indices; the author also needed equality between taking a minimum and the expression obtained after conversion back to a finite index. Subsequent retrieval examined unwrapping, coercion, and minimum interfaces, with some probed candidate names absent. Checking passed after explicit index equalities were supplied. The final file retains calls for stopping a submartingale, handling the negative process, and combining the properties into a martingale. [Appendix T.2](#) locates receipt of the advice, the author’s revisions, and final use.

Material entered the proof in both cases. B-M3 connected function representations; B-M1 connected indices across interfaces. The cases explain how retrieval became usable, while the 29 explicit-use records give coverage under the common start rule. Each serves a purpose; these records have not been used to calculate overall retrieval efficiency by configuration.

5.3 What changed in the original problem after a diagnostic?

Of the first proof or interface diagnostics, 19 concern actual implementations and 14 concern temporary declaration queries or local trials. A temporary query may return both a usable declaration's type and reports that other names do not exist. An implementation error may require changing a proof intended for delivery. Following the same target is necessary to establish what changed afterward.

Repairing finite-sum and measurability problems. Second-batch A-L1 constructs innovations and their partial sums. The proof needs measurability relative to observation histories and integrability of the partial sums. Its initial implementation had problems with measurability rewriting, undetermined indices, and a mismatched finite-sum integrability name. The author read the failing locations, searched finite-sum integral and integrability declarations, made the finite-index type explicit, and used the appropriate interface. It also had to connect an observation-history projection expression to the current function, proving a function equality before applying the available measurability result. An incidental editing error and a zero-term integrability goal were repaired as well. The same task file eventually passed checking and retained the measurability and finite-sum integrability proofs. Progress is established by the diagnostic targets, corresponding edits, and later check of that file. [Appendix T.3](#)

A local trial did not reach the same endpoint. In the second-batch linear minimum-mean-square-error task B-L6, a support author tried to prove an integral identity for the square of a three-term linear combination. The initial diagnostic showed that a product-integrability call did not match the current target; other square-expansion goals remained as well. The helper tried changing the expression; a patch command was unavailable during the process, and the error reappeared afterward. It continued with constant-multiplication queries, simplification attempts, and direct conversion. A later version advanced product integrability, but the record contains no passing check of the complete three-term square-integral identity.

The helper's return identified checked Gaussian-moment and independence components. The final task file does contain a single-sensor quadratic-expression proof, and the whole run passed common acceptance. Those items concern their own targets and do not establish completion of the three-term trial. The trial remains classified as having no observed resolution in the followed record.

B-L6's first retrieval process, meanwhile, has confirmed adoption. The author continued through Gaussian-distribution and equality-in-distribution representations; final source uses retrieved interfaces to transfer integral properties of the known distribution to the noise variable, proving zero mean and the square integral. Another variance interface returned in the material was not used. Thus the run contains both adopted retrieval results and a local trial whose completion was not confirmed. [Appendix S.2](#) and [T.3](#)

A-L1's later checks and the same file support repair of the original goal. B-L6's final task completion does not supply the missing completion evidence for that local trial. Checking both the goal identity and subsequent code preserves this distinction. Section 3.3 supplies the first-batch diagnostic distribution, while this section explains work after specific second-batch diagnostics; the complete individual table is in [Appendix S.2](#).

5.4 How does locally checked code reach the author?

The first support authors for second-batch A-M2 and A-M3 both produced code that passed local checking. To continue, the main author also needed to obtain that code from the return and integrate it into its own workspace. The table locates the observed transfer states.

Location checked	Observed state in A-M2 and A-M3
Support workspace	Code passes local checking
First return	Patch is empty
File subsequently read by the author	Still contains the task input
Later work	After redelegation, a later candidate replaces the first support candidate and enters the author's workspace through a nonempty patch

The empty patch and the author's file locate the break: the first local result did not enter that file through this return. The later nonempty patch came from another candidate, so eventual completion cannot count as delayed adoption of the first support draft. The evidence records the transfer outcome without establishing the internal cause of the empty patch.

Both first support candidates were replaced. The retrieval table in Section 5.2 has only one replaced record because A-M3's first retrieval starts with an earlier construction need of the main author, whose subsequent results entered the final proof; the object replaced here is the first support candidate. Different objects are being followed, producing different counts. [Appendix S.3](#) retains patches, file correspondences, and source locations.

This check also explains how adoption can be assessed: verify the actual return, examine changes in the author's file, and locate use in subsequent proofs. H1 in Section 4.3 shows delivered results connecting mathematical dependencies; this section locates the stage before results have entered the author's workspace.

5.5 Turning tool problems into testable improvements

The preceding subsections follow proof needs. This subsection returns to all tool calls to examine recurring execution problems. Command lookup fails in 173 calls, including 49 with an outer exit code of 0. A call can contain multiple internal commands, so its actual return provides more information than one outer status.

Calls to `apply_patch` account for 66 lookup failures across all 33 runs and 64 sessions. A new session may attempt the same unavailable entry point again; this distribution makes accurate startup information a useful improvement to test. Lookup failure can also involve names or search paths; categories and sources appear in [Appendix S.4](#).

A startup tool-information card was completed from this finding. Separately, execution-time parameter rejections motivated a pre-execution parameter check. Each prototype has its own functional validation. [46]

Prototype	Problem addressed	Validation completed
Startup tool-information card	Show the author the actual available tool entry points at the beginning of work	Isolated-container checks of default entry points and changed search paths confirm that the card distinguishes these states
Parameter preflight	Identify nonconforming parameter formats or values before execution	Static replay of 3,896 inputs to two tools identifies 9 original rejections and accepts 3,887 valid inputs, matching the original parameter conditions

Validation of the information card establishes the accuracy of its tool information; static replay establishes agreement with the original parameter conditions. Effects of the card on agent attempts, total effort, and mathematical outcomes await an actual comparison under fixed conditions. Parameter validity is a separate pre-execution check. Prototype implementation, functional validation, and evidence scope appear in [Appendix S.4](#).

Following the same proof need connects retrieval, feedback, revision, and use, while locating problems in code transfer and tool execution. Whether relevant material connects to the current goal and whether local work reaches the author help explain different working processes beneath the same completion outcomes.

6 Discussion and conclusions: organizing mathematical work

The preceding analyses connect delivery outcomes, mathematical work, and execution processes. This section returns to the three questions in Section 1: how to assess completed results, how to establish the roles of review and collaboration, and whether judgments of progress rest on comparable objects.

6.1 Read completion and effort together

A common mathematical criterion allows different implementations to be compared against the same task goal. Proof content still differs after a pass: first-batch B supplied an optional direct-expansion route in the variance task, while C covered a broader Cauchy family in the inversion application. These add derivation and input scope respectively. Examining them explains what was actually delivered beneath identical completion labels. [Appendix R.1.1](#) and [R.1.3](#); [21,23]

A has the higher median solving time in both batches without a corresponding coverage advantage. This supports examining the effort consumed by the prescribed workflow. Task-level results also show that the lower-median configuration is not faster on every task: B and C each lead on some second-batch tasks. The reported time includes assistance, waiting, and repair that actually occurred during solving, measuring the effort of completing the work.

Some prompts and execution arrangements differ between batches, and the configurations combine model and organizational differences. Applying the same mathematical criterion makes outcome judgments comparable but does not remove differences in execution conditions. Timing is therefore compared by task within each batch first; cross-batch observations indicate whether patterns recur. The speed effect of an individual workflow component still requires holding other conditions fixed. [41,42]

6.2 Locate the contributions of review and collaboration in concrete work

Review can be examined through what it identifies, what it requires, and what changes. First-batch A-M2’s opinion identified the difference between raw- and central-moment assumptions, followed by the corresponding derivation and public-interface revision. Old L3 shows that a reviewer can notice broader scope and still decide it need not be completed. Explicit obligations affect acceptance decisions; counts of opinions or pass labels cannot distinguish these contributions.

Collaboration can be examined through what returns, how it is integrated, and where it is used. First-batch H1’s calls locate the support proof’s contribution to later tasks. Second-batch B-M1 connects receipt of advice, the author’s adaptation, and final library calls; empty-patch cases locate checked code that has not yet entered the author’s file. These observations support improvements at the respective stages, but without a solo-solving counterfactual they cannot be converted into minutes saved by collaboration.

Task scope and delivery usability are therefore concrete checks to implement. Filling missing work and bringing results to their users can each make a verifiable contribution. Review can also confirm existing work and support submission: first-batch main-comparison C-L1 received a readable passing opinion and submitted an unchanged candidate. This is an instance of checking entering a submission decision. [Appendix R.2.1](#)

6.3 How does historical progress correspond to comparable objects?

Section 4.4 shows mathematical content added across revisions. Counting that development also requires checking whether earlier and later opinions concern comparable targets and which code each evaluates. The existing investigation first selects broad comparisons with opinions directly bound to the intended candidates, then purposefully checks a subset for affirmative mathematical-target comparability. A task can contribute multiple earlier/later comparison relations, and relations can share endpoints. Relation and task counts are therefore reported separately; relations are not independent task samples. [\[10,32\]](#)

Historical material	Comparison relations	Tasks	Earlier passes	Later passes
Broad directly bound comparisons	461	248	116	324
Relations with affirmative target comparability and qualified objects	128	79	28	45

The broad set shows substantial label growth. Growth remains where target comparability is also affirmatively established. The second row is a qualified subset from purposeful checking, not a random sample of the first. Their differing trends cannot all be attributed to erroneous evaluation or target changes.

Which historical changes do detailed cases cover? A separate, purposefully selected queue of detailed anomaly investigations overlaps 163 of the 461 relations, with passes increasing from 32 to 33. In the other 298 relations, passes increase from 84 to 291. Detailed cases explain how particular problems arise, but cover only part of the growth in the broad collection. They therefore cannot establish that the discovered anomalies account for most historical growth or estimate their prevalence across all tasks. [Appendix R.3.5](#)

Task composition in later revisions. Repeated-revision statistics are also affected by task composition. Follow-up fixes failure starting points and checks subsequent targets and corresponding code at each step. A first step is relative to such a start, and a task can contribute multiple starts. In the strictly qualified material, first steps pass in 22/69 cases and later steps in 3/31. The 31 later steps come from paths that

have not yet passed and still have qualified evidence, covering 15 tasks. Only 1 of the 19 corresponding first steps for those tasks passed. Thus tasks entering later follow-up already had a lower pass proportion in their observed first steps. Testing the return to further revision requires comparable tasks and conditions; these proportions are not a decay curve for the same task population. [Appendix R.3.6](#)

Saved code supplies another direct check. Existing research froze 14 file pairs: 6 development pairs and 8 separately selected pairs. Of 28 files, 17 compile and 16 yield the required named declarations. All 16 extracted targets pass the two saved axiom rules; the additional compiling file lacks the requested declaration. Compilation, presence of a named target, and proposition semantics answer different questions. For example, an early proof simply returns its conclusion supplied as a premise. Later source removes that shortcut but fails to compile in the saved environment. Both the source-level improvement and the saved-environment build failure should be reported. [11]

Information from additional rule comparisons. The same study also evaluated two candidates under two rule sets. For the tested axiom and direct-premise properties, the rules were identical or had known implication relations, and the additional combinations answered no more questions than a fair fixed-standard comparison already using those relations. This is a completed, bounded negative result. Constructed scenarios separately validated the program without adding real-task successes. [Appendix R.3.7](#) retains selection, all statuses, and the comparison basis.

This also distinguishes the roles of the historical investigation and Section 5. Historical cases follow longer proof development and caller changes; fixed second-batch starts cover early processes in every run. Purposeful historical selection and uniform starts answer questions within their respective scopes. The report uses both to explain the project while preserving the scope of their counts and conclusions.

6.4 Conclusions and directions for improvement

The report shows that probability proofs usable by later tasks must connect the original task conditions, actual derivations, and callers. A-M2 moves the centered-moment conversion inside the proof, H1 connects inversion to uniqueness and applications, and historical revisions complete models, limits, and calling interfaces. These changes constitute inspectable mathematical progress. Completion and effort across the two experimental batches also show why the prescribed workflow’s benefits need to be examined at individual stages.

The roles of review and collaboration are visible in specific decisions and subsequent work: identifying missing scope, requiring a derivation, confirming existing results, returning code, and integrating it. Retrieved material requires checking applicability and, where needed, adaptation before it becomes part of the current proof. B-M3 and B-M1 show how existing theorems connect different representations, A-L1 records actual repair following diagnostics, and B-L6 retains a local trial whose completion was not confirmed. These process explanations are more specific than tool-call counts and preserve the observed outcomes of different attempts.

The evidence points to three priorities: check public-theorem scope explicitly during review; check patches, receiving files, and actual use when assistance returns; and provide accurate tool-entry information at session startup. The first two address observed omissions and transfer breaks; the information card has passed functional validation. Evaluation should retain the common mathematical criterion and report mathematical outcomes alongside actual effort, including waiting, repair, and assistance. A model-outcome comparison of the card remains future work. A dedicated study of retrieval and recovery after proof difficulties would additionally need consistent starts, failures, adjustments, and endpoints in existing logs, including successful, replaced, abandoned, and unconfirmed processes.

On evaluation reliability, acceptance still increases in the historical subset with comparable mathematical targets and qualified objects. Lower pass proportions in later revisions accompany changes in task composition and cannot directly establish declining returns to revision. Saved-code checks further show why build and axiom-rule passes must be accompanied by examination of statements and derivations. These completed analyses support the report’s approach: establish what was proved, how results entered use, and which code and records support judgments of progress.

7 References and public project materials

The public repository provides the workflow implementation, Lean corpus, build instructions and selected historical cases. It also includes an isolated [workflow demonstration](#) that uses the production task-preparation, build, review-validation and application functions. Its small definition-and-caller example exercises an interface repair, a caller build failure and rejection of an outdated review. Lean compilation is real; default semantic opinions are recorded teaching reviews, with an option to configure an external reviewer. This is a software inspection entry point, separate from the probability-task evaluation. [36]

Numerical references below lead to the relevant technical-appendix sections. Appendices C and D contain historical methods and results; G and L contain detailed trajectories and mathematical arguments; M and N contain measurements and evaluation methods; O explains configuration differences, the causes of interrupted runs and the A-L7 continuation. [Appendix E.2](#) identifies which original records accompany the full evidence package and which require archive access. The retained [evidence guide](#) and browser entry point locate supplied requests, opinions, code and source maps; later verification records also accompany the package. Experimental materials are provided for this private review.

Ref.	Material	Technical appendix
[1]	Successive mathematical progress in inversion	L.5; A.2
[2]	Editing, semantic review and registration	A, B
[3]	Early orchestration and code editing	A, L.1
[4]	Gamma-to-Dirichlet proof responsibilities	B.4, L.2.1
[5]	Proof obligations and parent reassessment	B.1-B.3, L.2.2
[6]	Mathematical gates and MGF proof assembly	B.10, L.2.3
[7]	Total variation and its consumers	L.2.4
[8]	CLT and coupon-collection chain	L.3
[9]	Target changes and review objects	B, L.4
[10]	Historical comparisons, failing origins and selection	C
[11]	Saved-code checks, definition witnesses and rule comparison	D
[12]	Configurations and common task instructions	K, N.1
[13]	Corrected acceptance and current panel	K.8-K.9, M
[14]	Per-run durations, dispatches and tokens	H, M, N.5
[15]	Evaluator repair and regression checks	K.9, N.6
[16]	Feedback semantic coverage and evidence index	G.18, N.4
[17]	H1 actual reuse, support and recovery	G.13
[18]	B/C-M1 adoption, checking and documentation	G.11
[19]	Assistance production, delivery, recovery and adoption boundaries	G.1-G.4, G.18
[20]	C-L1 readable review and submission	G.8
[21]	Corresponding L2 calls and consistent interpretation	G.15
[22]	L7 control, condition repair, proof and disclosure	G.16
[23]	L3 tail and H1 textbook scope	G.13.1, G.17, K.8
[24]	Raw tool returns, process classification and co-occurrence	M.2-M.3
[25]	M2 raw-to-centered-moment proof repair	G.14.1; R02, R03, R14, R18
[26]	Previously captured local architecture, workflow and production contracts	A.5, K, N; V01-V06, P01-P02
[27]	L1 same-task mathematical routes, control, information and later revisions	G.5-G.10; N01-N13, R06-R08
[28]	M3 full-task adoption and confidence-format adjudication	G.12; U05-U06
[29]	Intermediate L3 premise removal and version-bound repair	G.14.2; R02, R03, T02
[30]	Common-evaluation rules, visible inputs and actual session settings	N.6; E.2; supplied evaluation evidence
[31]	Run selection, shared inputs, actual allowances and missing usage	N.1, N.5; E.2; supplied run evidence
[32]	Historical target decisions and saved-code sampling rule	C.4, D.1; E.2; supplied historical-method evidence
[33]	Innovation review-recovery records and unresolved attribution	G.6; E.2; supplied recovery evidence

Continued on next page

Ref.	Material	Technical appendix
[34]	Documented task lifecycle, repair, scheduling and canonical-state maintenance	A.5, E.3; public architecture, workflow, workspace-state and dependency documentation
[35]	Textbook–Mathlib interfaces, dependency selection and proof responsibility	E.3; interface policy, dependency trail and semantic-review criteria
[36]	Public implementation and isolated workflow demonstration	E.3; architecture and workflow-demo documentation
[37]	Candidate-state and six-service source correspondence	N.2.1, G.18, E.2; Revision 12 evidence guide
[38]	Independent audit of A/B/C versions, tools and information access	O.1
[39]	13 September A-L7 continuation and L3 review-responsibility audit	O.2–O.3
[40]	Stepwise A-L3 review and application-gate audit; C-L1 information and submission	O.4; G.8–G.9
[41]	Second-batch runs, conditions and costs	Q.1
[42]	Common outcome reconciliation and caller checks	Q.2
[43]	Old L3 diagnostics and completed repairs	Q.4
[44]	Second-batch code and feedback analysis	Q.5
[45]	Second-batch tool feedback and proof development	S.1–S.3; 33-run coverage, fixed starts, provenance and review
[46]	Startup information and parameter preflight	S.4; command lookup failures, static replay and planned comparison